

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

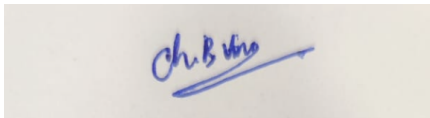
Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I Ch.B.Vinay(192425148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;
```

```

        default:
            printf("Invalid Choice\n");
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

- Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)
- Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)
- Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)
- Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)
- Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Answer:

a) Code Errors:

Code line	Error	Error Type
12	Missing ; after "Compiler"	Syntax Error
16	findLength() Without Argument	Semantic Error

18	Missing')'in if(count > 5	Syntax Error
27	in Case2 missing :	Syntax Error
37	marks[4]	Runtime Error
42	*ptr is used without initialization	Runtime Error
46	total / students where students = 0	Runtime Error
49	while(i <= 5) without increment	Logical Error

b) Error Classification

Error	Type
Missing ;	Syntax
Missing function argument	Semantic
Missing)	Syntax
marks[4]	Runtime
Missing :	Syntax
Uninitialized pointer	Runtime
Division by zero	Runtime
Infinite loop	Logical

(c) Compilation Phase

Error Type	Detected In
Lexical	Lexical Analysis
Syntax	Syntax Analysis
Semantic	Semantic Analysis
Logical	During Testing/Execution
Runtime	During Program Execution

(d) Correct Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int findLength(char str[])
```

```
{
```

```

    return strlen(str);
}
int main()
{ char name[20] = "Compiler";
  int count;
  count = findLength(name);
  if(count > 5)
  {printf("Length is greater than 5\n");
  }
  int marks[4] = {85, 90, 75, 80};
  printf("%d\n", marks[3]);
  int choice = 2;
  switch(choice)
  {
    case 1:
      printf("Option 1\n");
      break;
    case 2:
      printf("Option 2\n");
      break;

    default:
      printf("Invalid Choice\n");
  }

  int value = 50;
  int *ptr = &value;
  printf("%d\n", *ptr);

  int total = 100;
  int students = 5;

  float average = (float)total / students;
  printf("Average = %.2f\n", average);

```

```

int i = 1;

while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

(e) Three Compiler Features

1. Syntax error checking
2. Warning for uninitialized variables
3. Array bounds warning (compiler/static analyzer)

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```

#include <stdio.h>

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

```

```

int max = maximum(marks[0], marks[1]);

switch(max)
{
    case 80:
        printf("Maximum is 80\n");
        break;

    case 90
        printf("Maximum is 90\n");
        break;

    default:
        printf("Other Value\n");
}

do
{
    printf("%d\n", max);
    max--;
while(max > 75);

if(max == 75)
    printf("Limit Reached\n")
}

return 0;

```

Tasks

- (a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)
- (b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)
- (c) Correct the syntax errors and rewrite the complete program. (4 Marks)
- (d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)
- (e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

(a) Syntax Errors

Line	Error
3	Missing) in maximum() function.
18	Missing ; after printf().
21	Missing : after case 90.
32	Missing } before while.
32	Missing ; after while(max > 75).
35	Missing) and ; after printf().
39	Missing closing } for main().

(b) Classification

Error	Category
Missing)	Parentheses error
Missing ;	Statement terminator error
Missing :	Switch-case error
Missing }	Brace error
Incorrect do-while	Control structure error

(c) Corrected Program

```
#include <stdio.h>
int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}
int main()
{
    int marks[5] = {70, 80, 90, 85, 95};
    int max = maximum(marks[0], marks[1]);
    switch(max)
    {
        case 80:
```



```

        printf("Maximum is 80\n");
        break;
    case 90:
        printf("Maximum is 90\n");
        break;
    default:
        printf("Other Value\n");
    }
    do{
        printf("%d\n", max);
        max--;
    } while(max > 75);
    if(max == 75)
        printf("Limit Reached\n");
    return 0;
}

```

(d) Parser and Error Recovery

- The parser checks whether the program follows C grammar rules.
- Error recovery allows parsing to continue and reports multiple errors.

(e) Two Improvements

1. Display exact line number.
2. Suggest the missing symbol (;,), }).

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int factorial(int n)
{
    if(n <= 1)
        return 1;

    return n * factorial(n - 1);
}

```

```
int main()
{
    int @num = 5;

    int value = 10

    int result;

    result = factorial();

    if(result >= 100)
    {
        printf("Large Number\n");
    }

    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120
            printf("Factorial of 5\n");
            break;

        default:
            printf("Other Value\n");
    }

    int numbers[5] = {2,4,6,8,10};
    printf("%d\n", numbers[5]);

    int *ptr;
    printf("%d\n", *ptr);

    int a = 50;
    int b = 0;
    int c = a / b;

    int sum = result + 0 + value;

    int i = 1;
    while(i <= 5)
```

```

{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

- Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)
- Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)
- Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)
- Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)
- Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

(a) Errors

Line	Error Type	Reason
13	Lexical	Invalid identifier @num.
15	Syntax	Missing ; after int value = 10.
19	Semantic	factorial() called without argument.
31	Syntax	Missing : after case 120.
39	Runtime	numbers[5] is out of array bounds.
42	Runtime	Uninitialized pointer *ptr.
46	Runtime	Division by zero (a / b).
51	Logical	i is not incremented (infinite loop).

(b) Classification

Compiler Phase	Error
Lexical	Invalid identifier @num
Syntax	Missing ;, Missing :
Semantic	Missing function argument
Intermediate Code Generation	No error
Code Optimization	No error
Runtime	Array out of bounds, Uninitialized pointer, Division by zero

(c) Compilation Phases

Phase	Errors Detected
Lexical Analysis	Invalid token (@)
Syntax Analysis	Missing ;, Missing :
Semantic Analysis	Function call without argument
Runtime	Array index error, Pointer error, Divide by zero
Logical	Infinite loop during execution

(d) Corrected Program

```
#include <stdio.h>
int factorial(int n)
{
    if(n <= 1)
        return 1;
    return n * factorial(n - 1);
}
int main()
{
    int num = 5;
    int value = 10;
    int result = factorial(num);
```

```

if(result >= 100)
    printf("Large Number\n");
switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;
    case 120:
        printf("Factorial of 5\n");
        break;
    default:
        printf("Other Value\n");
}
int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[4]);
int x = 100;
int *ptr = &x;
printf("%d\n", *ptr);
int a = 50, b = 5;
int c = a / b;
int sum = result + value;
printf("%d\n", sum);
int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
    i++;
}
return 0;
}

```

(e) Two Improvements

1. Enable compiler warnings (-Wall).
2. Use static code analysis to detect runtime issues early.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors

Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand